COMSATS UNIVERSITY ISLAMABAD

ABBOTTABAD CAMPUS

SOFTWARE DESIGN & ARCHITECTURE

SUBMITTED TO: **Mr. MUKHTIAR ZAMIN**

PREPARED BY: **MUHAMMAD SHAYAN MUGHAL**

(SP23-BSE-050)

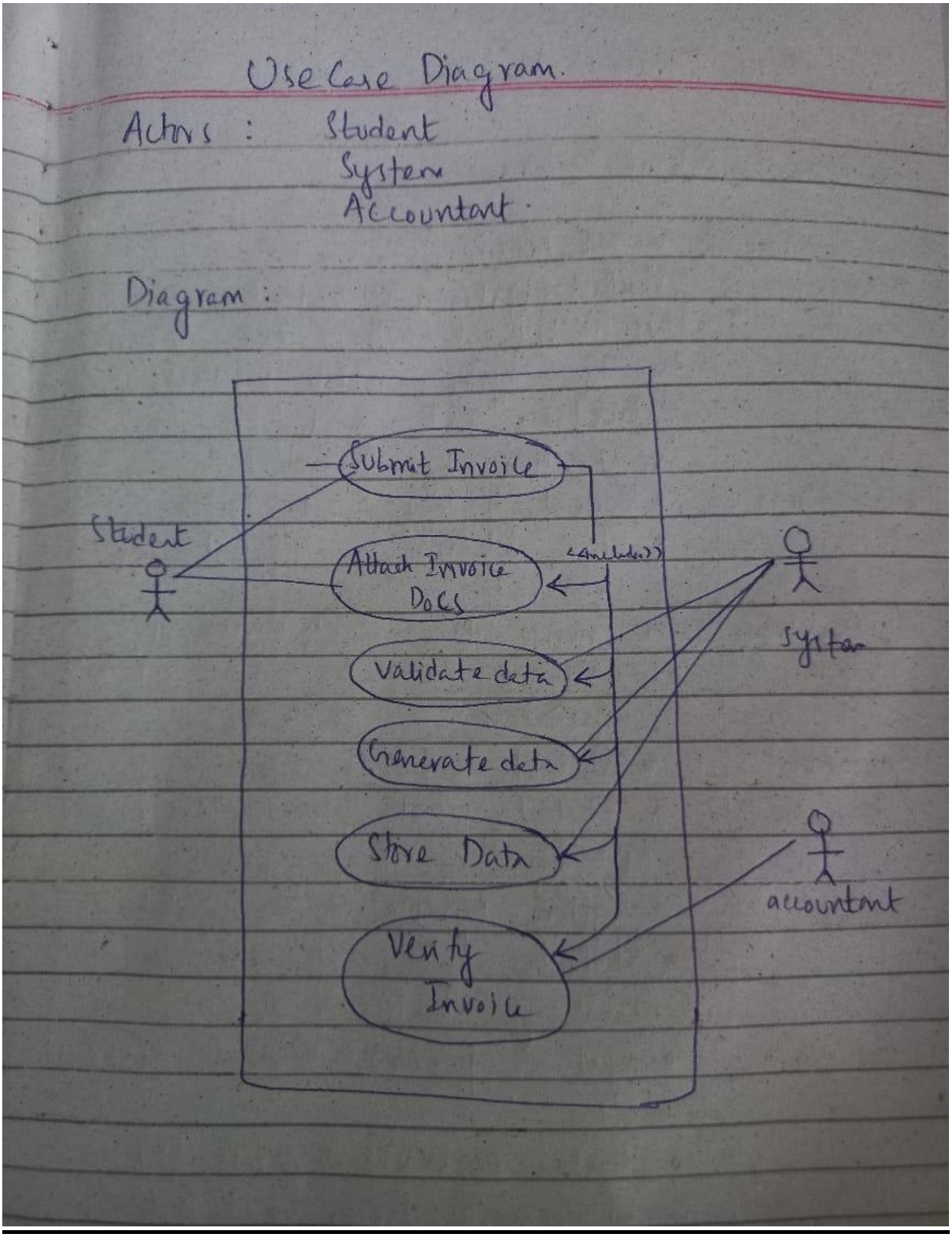
CLASS: **BSE-5B**

SUBJECT: **SEMESTER PROJECT**

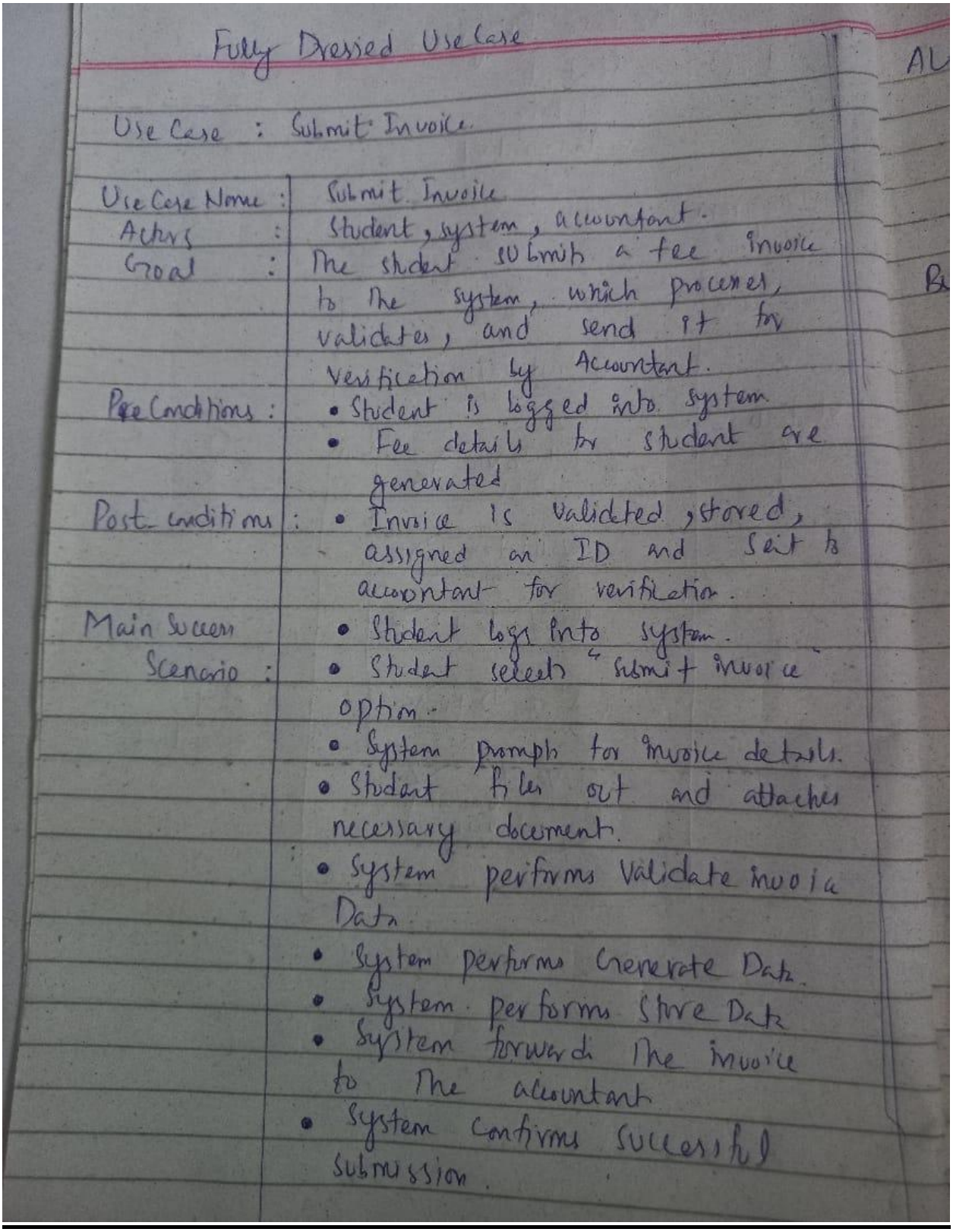
INCLUDES: **USE CASE DIAGRAM, FULLY DRESSED USE CASE, SSD, CLASS DIAGRAM etc**

COURSE: **SDA**

Use Case Diagram:



Fully Dressed Use Cases:



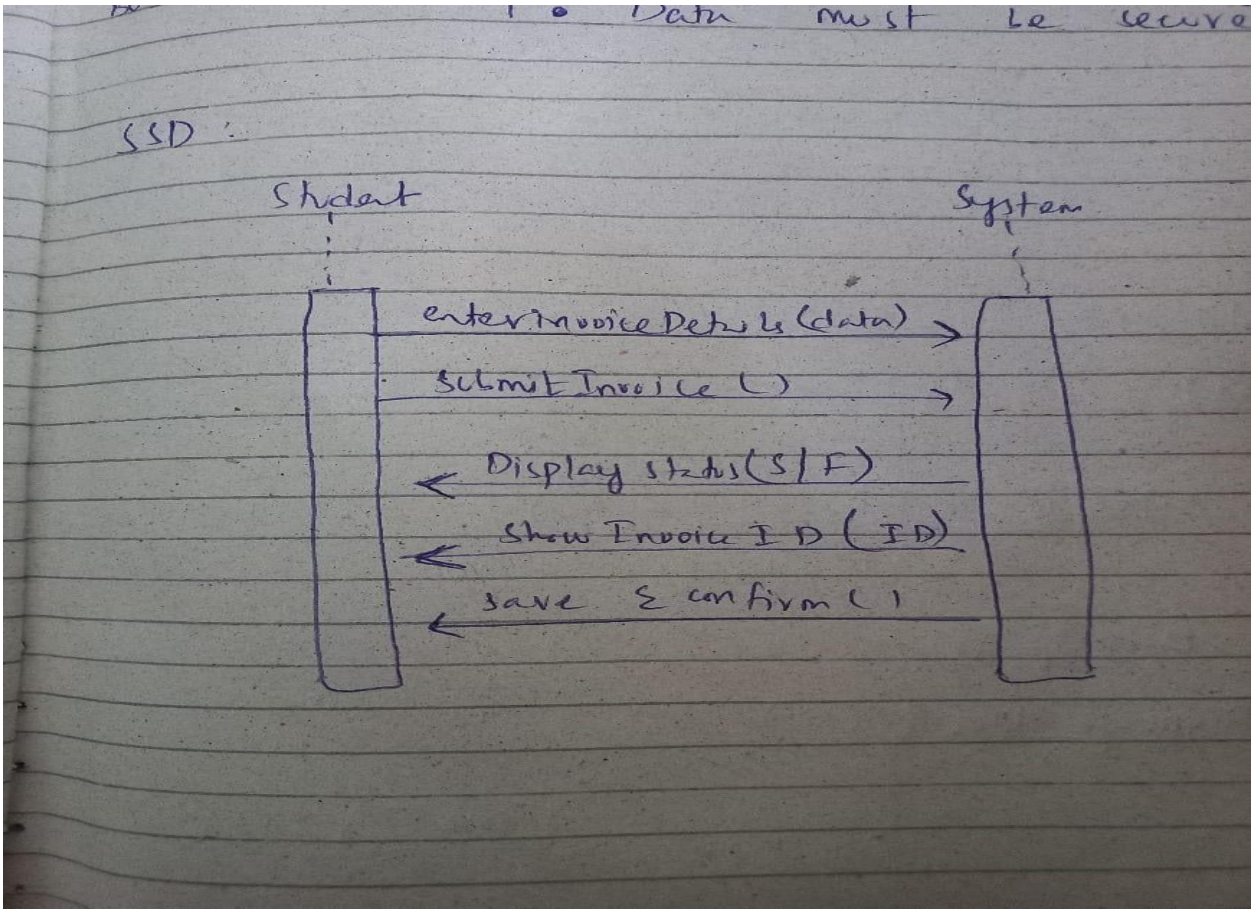
Fully Dressed Use Case

Use Case : Submit Invoice

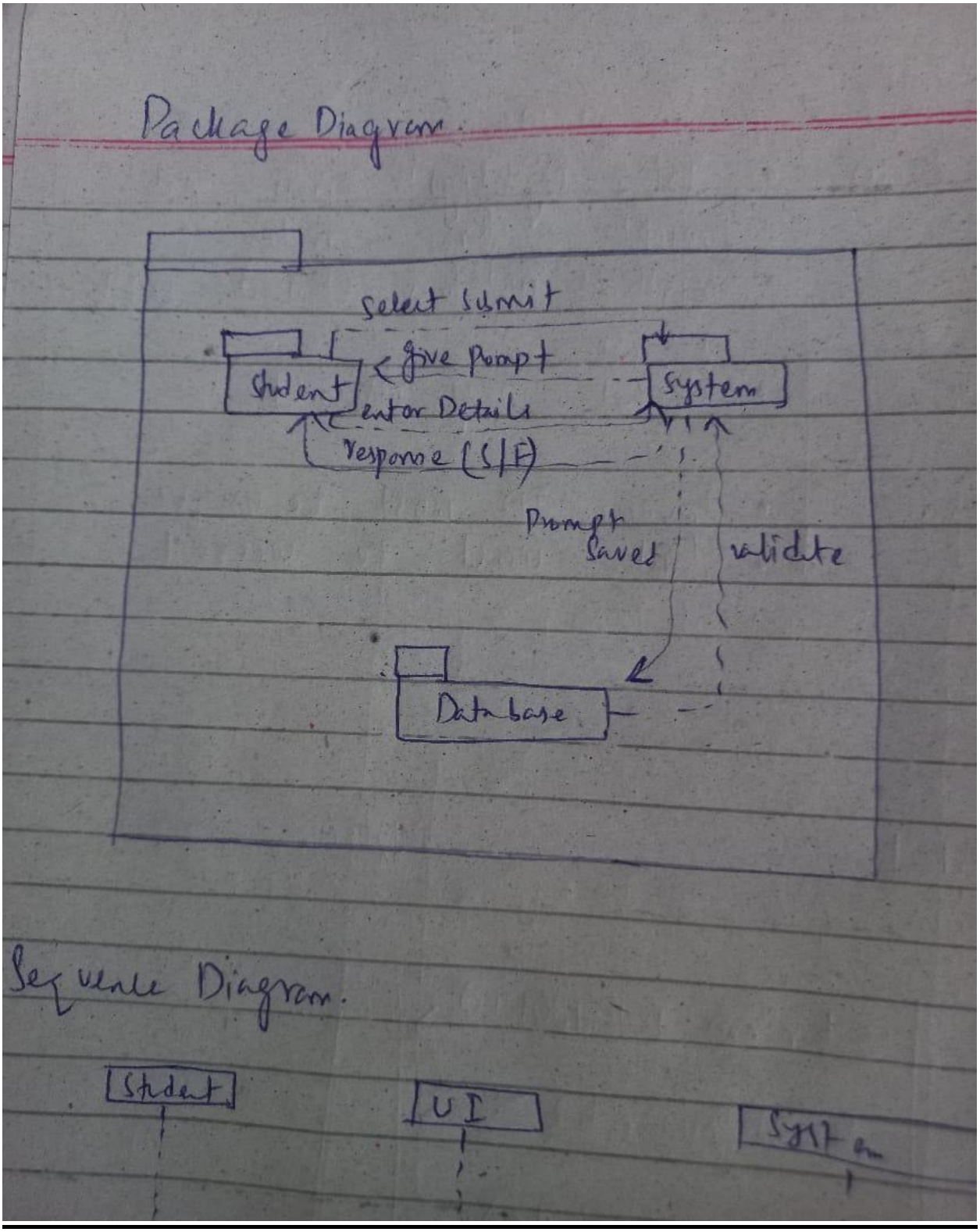
Use Case Name :	Submit Invoice
Actors :	Student, system, accountant.
Goal :	The student submits a fee invoice to the system, which process, validates, and send it for verification by Accountant.
Pre Conditions :	• Student is logged into system • Fee details for student are generated
Post conditions :	• Invoice is validated, stored, assigned an ID and sent to accountant for verification.
Main Success Scenario :	• Student logs into system. • Student selects "submit invoice" option. • System prompts for invoice details. • Student fills out and attaches necessary document. • System performs Validate invoice Data. • System performs Generate Data. • System performs Store Data • System forwards the invoice to the accountant • System confirms successful submission.

Alternative Flow	• System highlights errors and prompts for correction. • System notifies student with reason. • Student may resubmit.
Busines Rqs.	• Invoice ID must be unique. • Data must be secured.

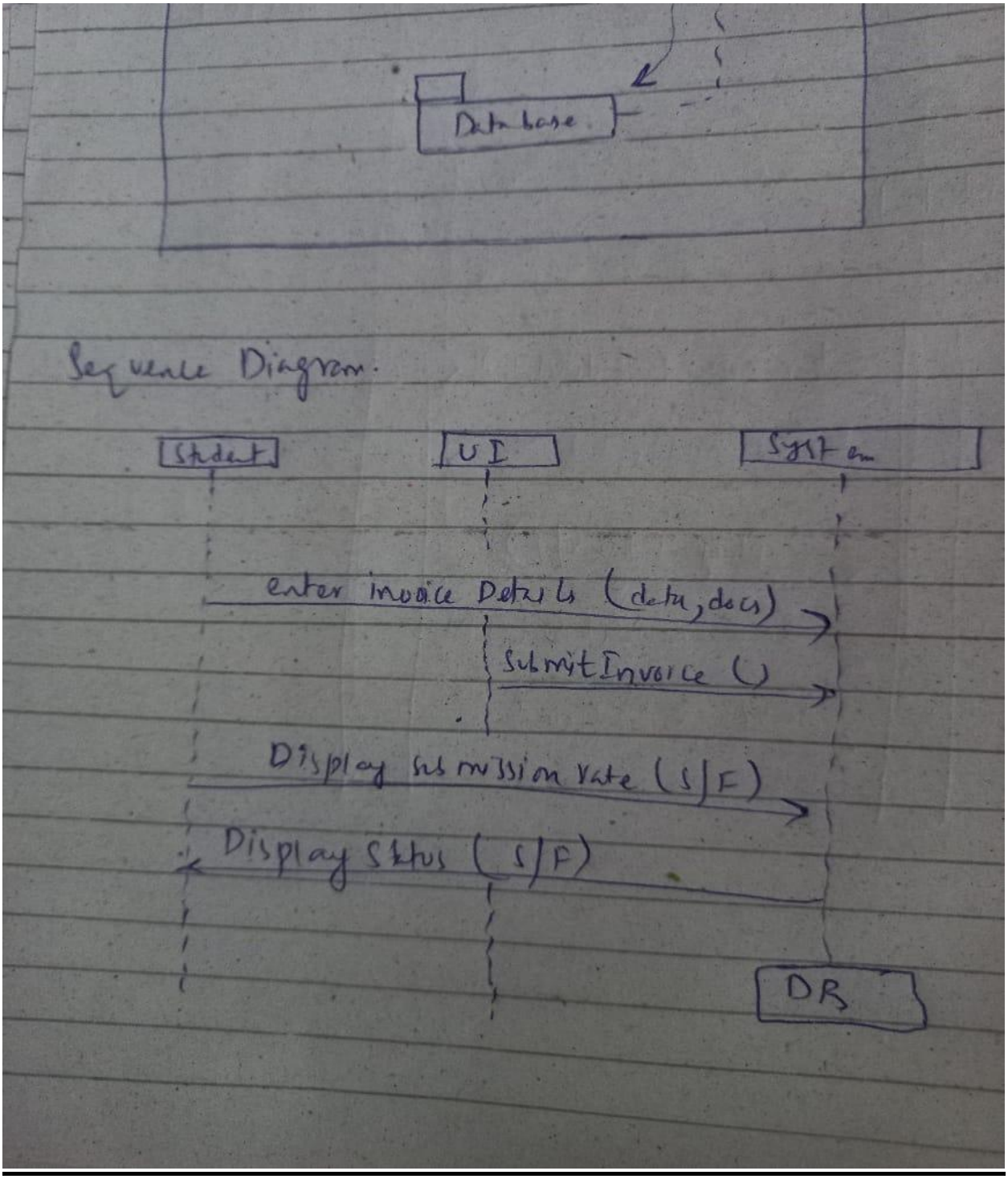
SSD:



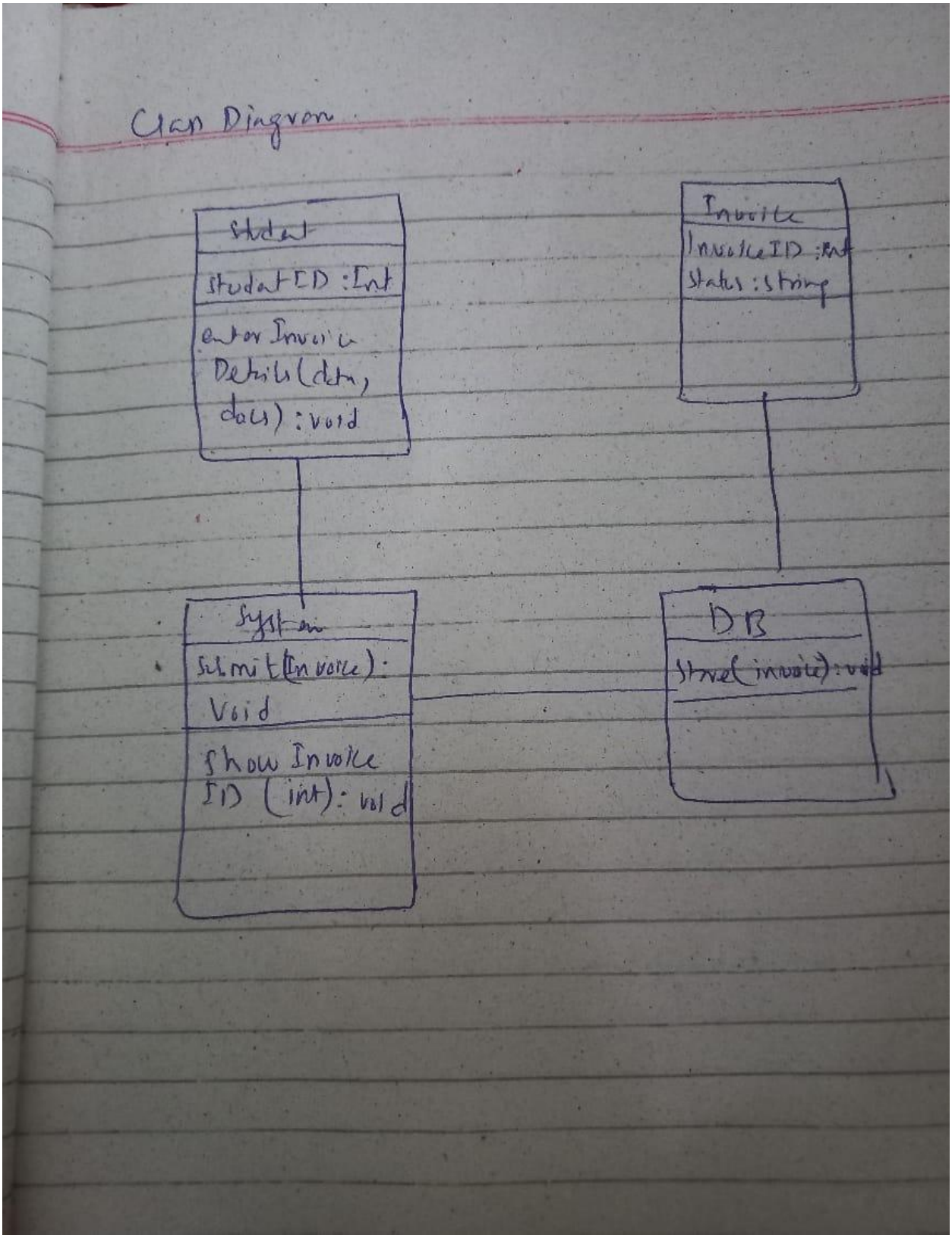
Package diagram:



Sequence diagram:



Class Diagram:



Coding Standards:

1. Naming Conventions

Element	Convention	Example
Project Name	PascalCase	InvoiceSystem
Package/Module	lowercase	invoice, services
Class Names	PascalCase	Invoice, InvoiceManager
Method Names	camelCase	submitInvoice(), validateInvoice()
Variable Names	camelCase	invoiceDate, clientId
Constants	UPPER_CASE	MAX_INVOICE_AMOUNT

---

2. Folder/Package Structure

```
bash
CopyEdit
invoicesystem/
├── models/              # Data classes (e.g., Invoice)
├── services/            # Business logic (e.g., InvoiceService)
├── repository/          # Data handling (e.g., InvoiceRepository)
├── ui/                  # Console/GUI interface (e.g., SubmitInvoiceForm)
└── utils/               # Helper methods (e.g., InvoiceValidator)
```

---

3. Code Formatting

- **Indentation**: 4 spaces (no tabs)
- **Line Length**: Maximum 100 characters
- **Braces**: Open on the same line (Java style)

```
java
CopyEdit
if (isInvoiceValid) {
    submitInvoice(invoice);
}
```

---

4. Error Handling

Use try-catch for all operations that may throw exceptions, especially during repository access.

```
java
CopyEdit
try {
    invoiceRepository.save(invoice);
} catch (DatabaseException e) {
```

```
    System.err.println("Error saving invoice: " + e.getMessage());
}
```

---

5. Validation and Security

- Validate all inputs: null checks, format checks (e.g., invoice date, amount).
- Sanitize input fields to avoid injection attacks.
- Keep class fields private; use getters/setters for access.

```
java
CopyEdit
public class Invoice {
    private String invoiceId;
    private double amount;

    public String getInvoiceId() {
        return invoiceId;
    }

    public void setInvoiceId(String invoiceId) {
        this.invoiceId = invoiceId;
    }
}
```

---

6. Testing Standards

- Write test cases for key methods: `validateInvoice()`, `submitInvoice()`.
- Use mock objects for testing repository operations.